

GeoGuide Project Guide

What GeoGuide is, how every part works in plain words, why each tool was chosen, how to run it on your own machine, and how to demo it, with answers to the questions people usually ask.

Local: `localhost:5173` Code: github.com/varshakodi/GeoGuide

Start here

If you only read one section, read this one.

The problem. A traveller arrives in a city and wants to know what is around them, what is happening on their dates, and what to watch out for. Travel apps are generic, and AI chatbots sound confident even when they are making things up.

What we built. GeoGuide knows where you are and which date you are travelling. It gives you a briefing (weather, events, safety, culture, history, top places), shows places and hotels nearby, suggests what fits your next 30 to 90 minutes, and answers questions.

Every sentence it shows comes with a citation you can tap to see the exact database row it came from.

The mandatory enhancement (Date-Shift Briefing). Change the travel date and everything is recomputed from the data for that date: events, weather, season, advisories and closures. A "What changed" panel lists exactly what moved.

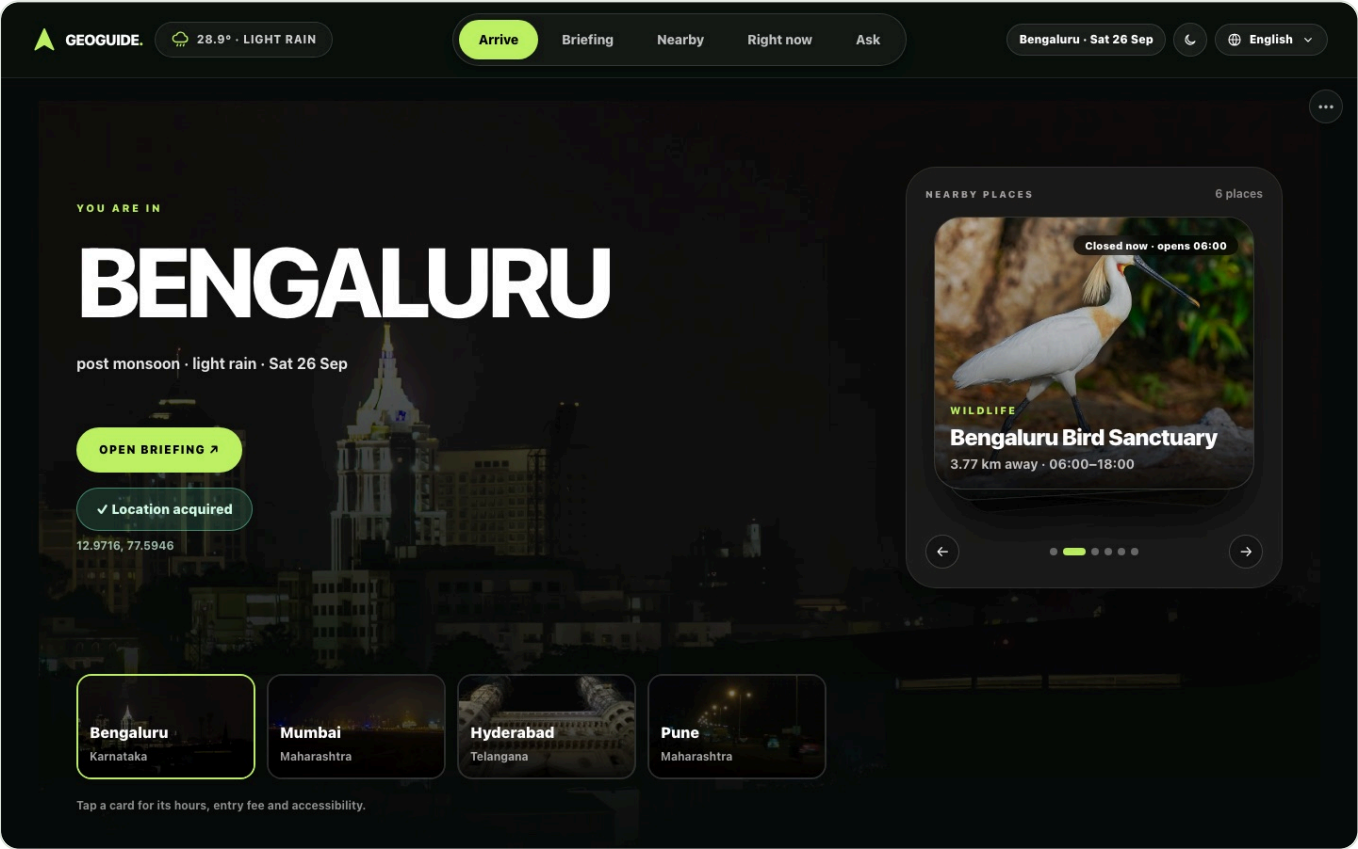
OUR ONE LINE

"Every sentence has a receipt." GeoGuide either answers from a real source it can show you, or it says it doesn't know.

60 cities in the dataset	3,173 + 900 guide passages + place facts we can search	3 grounding layers, plus a citation check	0 AI calls needed for Date-Shift facts
68 automated tests, all passing	1 Sep – 30 Oct 2026 the 60 days the data covers		

Screens

Captured from the running app (Bengaluru, dark theme).



Arrive: location, weather and nearby places

BRIEFING · SAT 17 OCT

Bengaluru

12 claims · 12 cited · 0 uncited dropped

ON NOW · Monsoon Music Nights

Read aloud

TIME TRAVEL

Move through your trip.

60 days in the GeoGuide dataset · 5 events flagged

THU 8 OCT

FRI 9 OCT

SAT 10 OCT

SUN 11 OCT

MON 12 OCT

TUE 13 OCT

WED 14 OCT

THU 15 OCT

FRI 16 OCT

SAT 17 OCT

SUN 18 OCT

MON 19 OCT

TUE 20 OCT

WED 21 OCT

THU 22 OCT

FRI 23 OCT

SAT 24 OCT

SUN 25 OCT

MON 26 OCT

Event window

Selected day

Monsoon Music Nights

Boat Race

Bengaluru Film Festival

Kite Festival

Drag or swipe to explore

WHAT CHANGED · RECOMPUTED FOR BOTH DATES

Sat 26 Sep → Sat 17 Oct

6 changes

EVENT

Monsoon Music Nights is on (17–22 Oct)

events_festivals evt_c46d368e

SEASON

Peak travel season: expect more visitors and higher prices

cities Bengaluru (peak_months)

WEATHER

Sky: light rain → partly cloudy

weather_daily wth_bb28443d (condition) +1

WEATHER

Rain: 5.4 mm → 1 mm

weather_daily wth_bb28443d (precipitation_mm) +1

Briefing, shifted to Sat 17 Oct: the festival is on and the "What changed" panel appears

WHAT CHANGED · RECOMPUTED FOR BOTH DATES

Sat 26 Sep → Sat 17 Oct

6 changes

EVENT

Monsoon Music Nights is on (17–22 Oct)

events_festivals evt_c46d368e

SEASON

Peak travel season: expect more visitors and higher prices

cities Bengaluru (peak_months)

WEATHER

Sky: light rain → partly cloudy

weather_daily wth_bb28443d (condition) +1

WEATHER

Rain: 5.4 mm → 1 mm

weather_daily wth_bb28443d (precipitation_mm) +1

WEATHER

Daytime high: 28.9 °C → 30.3 °C

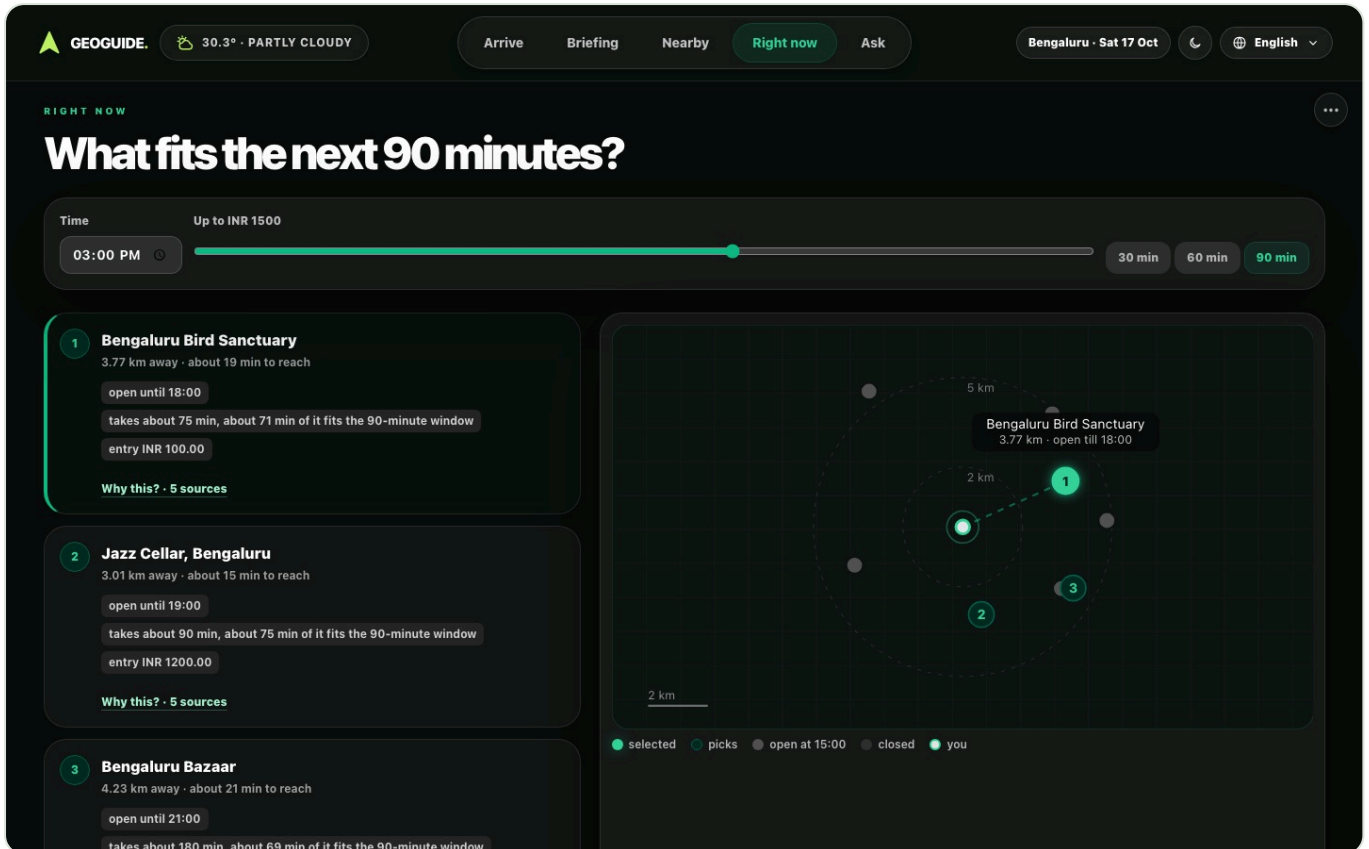
weather_daily wth_bb28443d (temp_max_c) +1

WEATHER

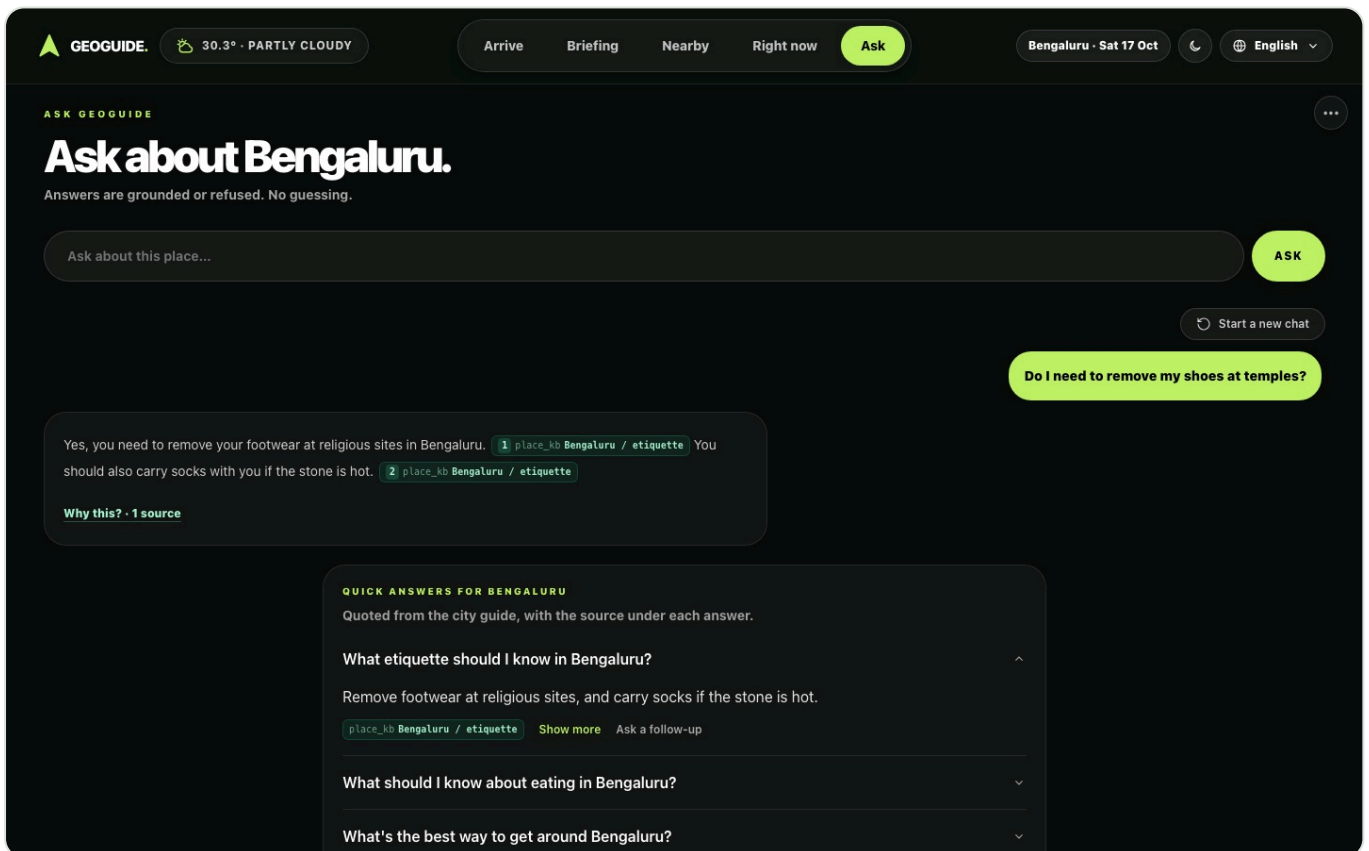
Humidity: 68% → 79%

weather_daily wth_bb28443d (humidity_pct) +1

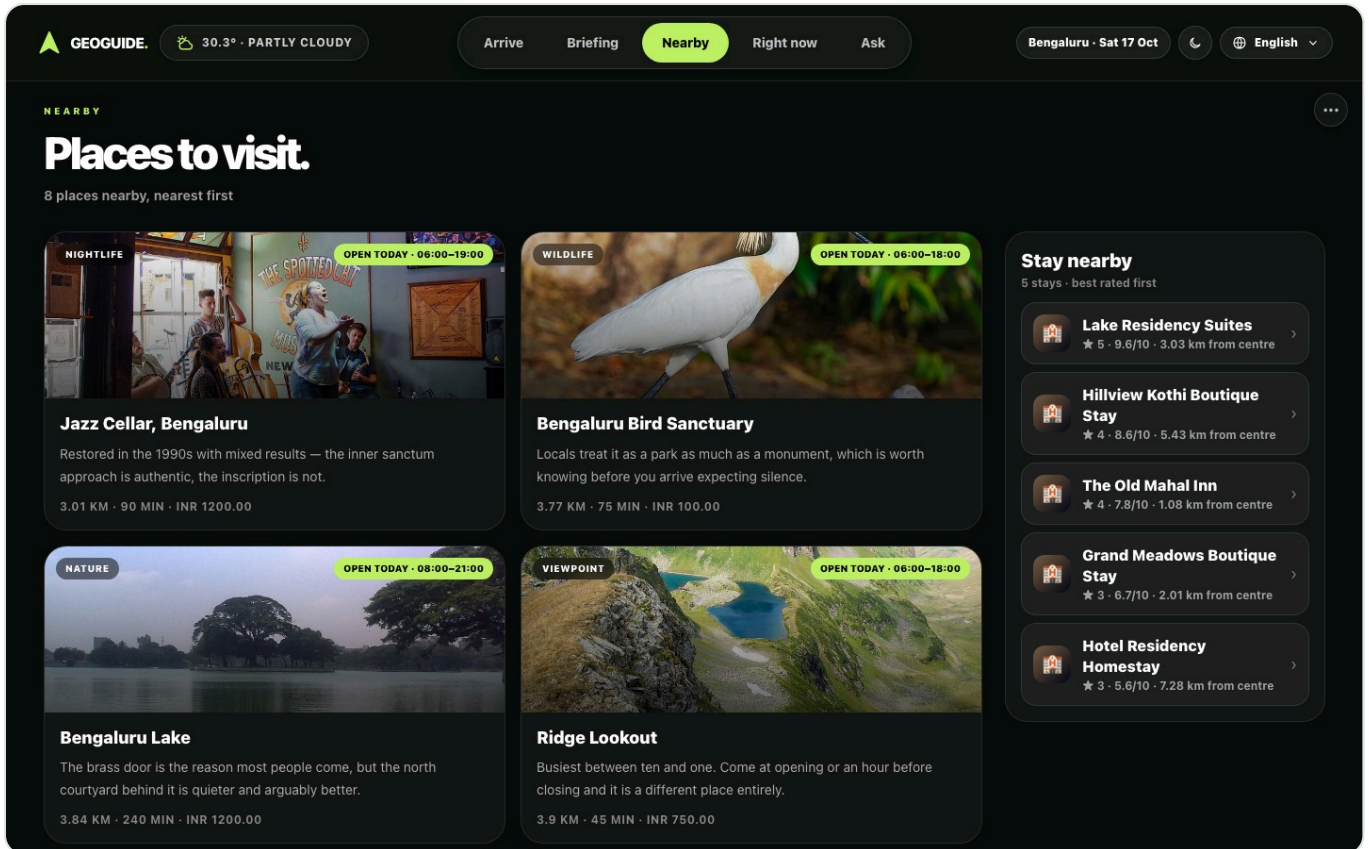
What changed: each difference cites the rows it compared



Right now: picks that fit the window; the selected one glows on the radar



Ask: an answer with numbered citations



Nearby: places first, places to stay beside them

What GeoGuide does

Five tabs, plus a few features that run across all of them.

FEATURE	WHAT THE USER SEES	HOW IT WORKS, IN ONE LINE
Arrive	"You are in Bengaluru", today's weather, nearby places as swipeable photo cards ("Closed now · opens 06:00")	Browser location → nearest city in our database → that city's data for the date
Briefing	Six cards: what to wear, what's on, safety, history, culture, top attractions. Each sentence has a numbered citation	Gemini writes the cards from retrieved passages only; sentences without a valid citation are dropped
Date-Shift	A date strip; move it and the facts and the "What changed" panel update	Plain database queries for each date; no AI involved
Nearby	Places to visit (nearest first) and hotels in a "Stay nearby" column; tap for details	Distance from you to every place, sorted; hotels sorted by guest score
Right now	"What fits the next 30/60/90 minutes?" Top picks with reasons, a radar map	Filters out places that are closed, too far or over budget, then scores what's left
Ask	A chat that answers with citations or politely refuses; quick FAQ answers	Search our passages → Gemini answers only from them → citation check
Evidence drawer	Tap any citation chip to see the actual database row, with the cited field highlighted	The chip's label is looked up by its ID in the read-only database (/source)
Languages	English, Hindi, and the city's own language (Kannada for Bengaluru)	Languages come from the database; Sarvam AI translates
Read aloud	A red button that speaks the briefing, highlighting the sentence being read	The browser's built-in speech voice

FEATURE	WHAT THE USER SEES	HOW IT WORKS, IN ONE LINE
Accessible briefing	The round blue button: a simple, large-text briefing with read aloud	Same briefing, simpler layout
Offline	An amber bar if the connection drops, while saved briefings and places still show	Every successful response is saved in the browser and reused when the network fails

How it works, end to end

What happens between opening the app and tapping a citation.

1 Your browser gives us coordinates

With your permission (HTTPS only), or you tap a city card. We don't store them.



2 The React app asks our FastAPI backend for that place and date

`/context?lat=...&lng=...&for_date=...` . The backend finds the nearest city by straight-line distance.



3 The backend reads the facts from the database

Weather row, events on that date, advisories valid at noon, places closed that weekday. SQLite, opened read-only.



4 For the briefing and chat, it searches our passages

ChromaDB finds the 4 guide passages closest in meaning to the question, for that city only.



5 Gemini writes the answer from those passages only

Each sentence must end with a citation like `[2]` pointing to a passage it was given.



6 We check the citations

Sentences without a valid citation are removed. If nothing is left, we refuse.



7 The app shows each sentence with its chip

Tap it and `/source` fetches the exact row or passage behind it.

SAY IT SIMPLY

"The browser gives us coordinates, we match them to the nearest city, pull that city's facts for your date from the database, and the AI can only write from what we retrieved, with a citation on every sentence."

How we stop the AI making things up

This is the heart of the project. Know it well.

The problem: large language models (like Gemini or ChatGPT) sometimes *hallucinate*, stating false things confidently. For a travel guide that's dangerous: a wrong opening time, or a festival that isn't on.

Our approach is RAG (Retrieval-Augmented Generation). Think of an open-book exam: before the AI answers, we find the right pages in our own book (the PS-13 guide texts and database) and hand only those pages to the AI. It must answer from them and cite them. It is not allowed to use anything else it "knows".

The checks, in order

STEP	WHAT IT CHECKS	IF IT FAILS
Layer 3 · intent	Is this a question we can never answer from our data? Live cab fares, exchange rates, traffic, "book me a room", a named official's phone number, or "ignore your rules and guess". Works in English and Hindi.	Refuse immediately, before any AI call. E.g. "I don't have live data such as fares, exchange rates or traffic, so I won't guess."
Layer 1 · relevance	Search our passages for this city. Is the best match similar enough? (similarity score at least 0.30)	Refuse, without calling the AI.
Generate	Gemini gets the question plus the 4 best passages, numbered, and must cite [n] on every sentence.	—
Layer 2 · "not enough info"	The AI is told to reply with a special code word if the passages don't contain the answer.	Refuse.
Citation check	Every sentence must cite a passage number that exists.	Drop that sentence. If none are left, refuse.

Why layer 3 runs first: we measured that some trick questions match our passages well. "How much is a cab to the airport right now?" lands close to our transport passage. A similarity score alone can't catch it, so rules catch those first.

Made-up places: if the question names something no retrieved passage mentions (like "Tipu Glass Tower"), the fallback path refuses instead of answering about something else.

If Gemini is down or slow: we don't invent anything. The app quotes the source passage word for word and labels it "Quoted verbatim · AI model offline". Date-Shift facts never

need the AI at all.

The grounding switch (the proof): the Judges view (the **...** button) has a "Grounding" toggle. When it's off, search returns nothing, so layer 1 refuses every question. If the AI were answering from its own memory, it would still answer. It doesn't.

How we chose 0.30: we tested 56 questions (37 real travel questions, English and Hindi, and 19 trick questions) and picked the score that best separates real from trick questions. It's written up in `docs/calibration.md`.

SAY IT SIMPLY

"The AI can only use what we retrieved. Every sentence needs a citation. Anything it can't back up is dropped or refused. And we never let it guess."

Date-Shift, the mandatory enhancement

What changes when you move the travel date, and why it's always correct.

WHAT CHANGES	HOW WE DECIDE IT FOR THE CHOSEN DATE
Events	Events whose start date $\leq$ your date $\leq$ end date. If none: "Nothing is scheduled in Bengaluru on Fri 25 Sep", plus the next event.
Weather	That day's weather row: condition, temperature range, rain in mm, humidity. Tips follow from it (rain $\geq$ 10 mm $\rightarrow$ rain jacket).
Season	The season on that day's weather row, and whether the month is one of the city's peak months.
Safety advisories	Advisories valid at 12:00 noon (India time) on that date, compared as real date-times with time zones.
Closures	Places whose closed days include that weekday (e.g. closed on Sundays).

No AI in this part. It's plain database lookups, so it's instant, repeatable, and works even if Gemini is down. Each fact cites its row (e.g. `weather_daily / wth_833a28a0 (precipitation_mm)`).

"What changed" panel: after you move the date, we compute both dates the same way and list only the differences, for example "Fri 25 Sep $\rightarrow$ Sat 17 Oct: + Monsoon Music Nights, + peak season, rain 10.65 $\rightarrow$ 1 mm". Each line opens both days' rows side by side.

Honest empty dates: when nothing is on, we say so and show a chip for the query that returned 0 rows. We never fill the gap with a made-up festival.

Dates outside the data (before 1 Sep or after 30 Oct 2026) are moved to the nearest date we have. The chat says what range the data covers instead of guessing.

The data

All from the PS-13 SQLite database the organisers provided. We don't scrape or add outside facts.

60 cities (India and abroad)	900 places (activities_poi)	300 hotels	300 events and festivals
3,600 daily weather rows (60 days × 60 cities)	400 safety advisories	1,200 guide passages, split into 3,173 searchable pieces	900 place facts

Rules we follow (and test)

- **Read-only:** the app opens the database read-only, so it can't change or add data.
- **Only active rows:** anything marked inactive is filtered out, never deleted.
- **Money is exact:** prices are kept as exact decimals with a currency code (INR), never rounded floats.
- **Time zones respected:** advisory start and end times are compared with their time zone.
- **IDs untouched:** IDs like `evt_c46d368e` are passed through as-is, and that's what the citation chips show.

Each rule has its own test (rules R1–R8 in `data-model/DATA_MODEL.md`).

Tech stack and why we chose it

Each pick was made for trust, speed, and running free on one laptop.

TOOL	WHAT IT IS	WHY WE CHOSE IT
React 18	A JavaScript library for building web pages from reusable pieces ("components")	Chips, cards and drawers are reused everywhere, and one date change updates every screen at once
Vite	The tool that runs and builds the React app	Starts instantly and reloads on every save, so we moved fast
Tailwind CSS	Styling written as short class names in the HTML	Consistent light and dark themes and phone layouts without a pile of CSS files

TOOL	WHAT IT IS	WHY WE CHOSE IT
FastAPI (Python)	A Python framework for building the backend API	All our AI tools are Python, so the API sits right next to them; fast, with automatic input checking
SQLite	A database stored in one file	The PS-13 data came as SQLite; opened read-only; no database server to run
ChromaDB	A "vector database" that finds text by meaning, not exact words	Runs locally and free; finds "remove shoes at temples" even if you ask "footwear rules"
MiniLM embeddings	A small model that turns text into numbers so similar meanings land close together	Multilingual (understands Hindi and English), small enough for a laptop CPU
Google Gemini (Flash)	The large language model that writes the sentences	Fast, cheap, follows "answer only from these sources" well, generous free tier; backup models if one is busy
Sarvam AI	An Indian-language translation service	More natural Hindi and Kannada than generic translators
Web Speech API	The browser's built-in text-to-speech	Read aloud for accessibility, no extra service or cost
Browser storage	Small storage inside the browser (<code>localStorage</code>)	Keeps the last data so the app works through a Wi-Fi drop
Vercel	Hosting for the web app	Free, deploys in a minute, gives HTTPS (needed for location)
Cloudflare tunnel	Gives our laptop's backend a public HTTPS address	Free and needs no server; for production we have a Docker and Render setup ready
pytest	Python's testing tool	68 tests prove Date-Shift, citations, refusals and data rules keep working after every change

IF ASKED "WHY THIS STACK?"

"Everything was chosen for trust and speed. Python keeps the AI and the API together, SQLite is opened read-only so we can't invent data, RAG with citations makes every answer checkable, and the whole thing runs free on one laptop."

Where things live in the code

One line per file. Each of us owns one area.

Frontend · `frontend/src/` · Vachana

- `App.jsx` : all five screens, the citation chips, the evidence drawer, the "What changed" panel
- `api.js` : every call to the backend, plus the offline cache
- `hooks/useGeoLocation.js` : asks the browser for your location
- `components/` : photo cards, map (radar), date strip, read aloud, accessible briefing
- `dates.js` : shows dates as "Fri 25 Sep"

Backend · backend/ · Vishnu

- `main.py` : starts FastAPI, sets which websites may call it (CORS)
- `ai_routes.py` : all endpoints (`/context` , `/briefing` , `/ask` , `/nearby` , `/now` , `/date-diff` , `/source` , `/health`)
- `data_queries.py` : every database query
- `ranker.py` : the Right now scoring

AI · ai/ · Vamika

- `pipeline.py` : the chatbot: layers → Gemini → citation check
- `retrieval.py` , `index.py` : search and the ChromaDB index
- `refusal.py` : layer 3 rules and refusal messages (English, Hindi, Kannada)
- `citations.py` : reads `[n]` , drops uncited sentences
- `briefing.py` , `cache.py` : builds and caches briefings
- `translate.py` , `session.py` : Sarvam translation, chat memory

Data & trust · Varsha

- `backend/date_facts.py` : Date-Shift facts and weather tips (no AI)
- `backend/date_diff.py` : "What changed" between two dates
- `backend/source.py` : turns a citation back into its database row
- `tests/` : 68 tests · `ai/evals/` : the 56-question evaluation set
- `data-model/seed/PS-13.db` : the provided database

How Right now picks places

For each place: skip it if it's closed that day, not open when you'd arrive, would close before you finish, is too far to reach with 20 minutes to spare, or is over your budget. Travel time is estimated at about 12 km/h (a mix of walking and autos). The rest are scored **45% how near**, **35% how much of the visit fits your window**, **20% how cheap**, and the top 3 are shown, each with its reasons and citations.

Run it locally

The backend stays on your machine; nothing needs a server. You need Python 3.11, Node 18+ and a Gemini API key (a Sarvam key is optional, for Hindi and Kannada).

First time only

1. Clone the repository and open a terminal in it.
2. Create the Python environment: `python3 -m venv .venv && source .venv/bin/activate`
3. Install the backend: `pip install -r backend/requirements.txt`
4. Add your keys: `cp .env.example .env`, then set `GEMINI_API_KEY`, `GEMINI_MODEL` and, if you have one, `SARVAM_API_KEY`. The `.env` file is git-ignored; never commit it or put keys in a `VITE_` variable.
5. Build the search index (about a minute): `python -m ai.index --reset`
6. Install the web app: `cd frontend && npm install`

Every time

1. Terminal 1, the API: `.venv/bin/uvicorn backend.main:app --port 8000`
2. Terminal 2, the web app: `cd frontend && npm run dev`
3. Open `http://localhost:5173`. Check `http://localhost:8000/health` shows `"status": "ok"`.

Before a demo

- ☐ Generate and cache the demo briefings so they load instantly: `python -m ai.prewarm --cities cty_17b8ef2f --dates 2026-09-25 2026-10-17 2026-10-24 --langs en-IN`
- ☐ Run the tests: `python -m pytest tests -q` (68 should pass).
- ☐ Laptop plugged in, sleep off, notifications silenced; browser zoom around 110%.
- ☐ Have a backup recording of the walkthrough ready.

SHARING A LIVE LINK

A public backend spends your API quota whenever anyone uses the chat or translation, and its grounding switch is shared by all visitors. For a portfolio, keep the backend local and show screenshots or a recording. If you ever expose it (for example through a Cloudflare tunnel), stop the tunnel afterwards and replace the keys.

Demo walkthrough

About eight minutes in Bengaluru. The three demo dates are cached, so every step is instant.

0:00–2:00

The problem, Arrive and the receipts

- One sentence on the problem, then the line: "Every sentence has a receipt."
- `Arrive` → "You are in Bengaluru". The browser gives coordinates, matched to the nearest city.
- `Briefing` → the pill "N claims · N cited · 0 uncited dropped".
- Tap a weather chip → the evidence drawer shows the database row, cited field highlighted.

2:00–4:00

Date-Shift

- Tap "Monsoon Music Nights" (Sat 17 Oct) → "What changed": + festival, + peak season, rain down.
- Pick Sat 24 Oct → "Monsoon Music Nights is over".
- Fri 25 Sep → "Nothing is scheduled", with a 0 rows chip.
- Say: "Recomputed from the database, not regenerated by the AI, so it's instant and always right."

4:00–6:00

Ask: grounded answers and refusals

- "Do I need to remove my shoes at temples?" returns an answer with numbered citations.
- "How much is a cab to the airport right now?" is always refused by the intent rules.
- Switch the language to Hindi or Kannada.
- Optional: turn grounding off in the Judges view (•••). Every question is refused, which proves the model isn't answering from memory.

6:00–8:00

Nearby, Right now, offline

- `Nearby` → places first, stays beside them; open a hotel and its source chip.
- `Right now` → set time and budget; select a pick and it glows on the radar.
- Optional: switch Wi-Fi off. The offline bar appears and saved data still loads.
- Close: "GeoGuide knows where you are and when you're going, and every sentence has a receipt."

Questions people ask

Short answers to say out loud.

About the product

What problem are you solving?

Travellers get generic guides, and AI chatbots that sound sure even when they're wrong. GeoGuide gives a briefing for exactly where you are and when you're going, and every sentence can be checked.

How is this different from Google Maps or ChatGPT?

Maps shows places but doesn't brief you for a date. ChatGPT answers everything, even when it's guessing. We only answer from our data, cite every sentence, and say "I don't know" when we don't.

What is the Date-Shift Briefing and why does it matter?

Change your travel date and events, weather, season, advisories and closures are recomputed from the data for that date, and "What changed" shows the difference. Travellers plan ahead, so the briefing must be for their dates, not today.

How does it know where I am?

The browser's location feature, with your permission, which works only over HTTPS. We match the coordinates to the nearest city in the database. You can also just tap a city.

Why can't it tell me cab prices or book a hotel?

Those need live data we don't have. Guessing a fare would be exactly the kind of made-up answer we're avoiding, so we refuse clearly and say why.

About the AI

How does the chatbot work?

RAG: we search our guide passages for the 4 closest in meaning to your question, give only those to Gemini, and it must cite one on every sentence. Uncited sentences are dropped.

How do you stop hallucinations?

Three layers: rules refuse questions we can't answer, a relevance check refuses when nothing matches, and the AI says so when the passages don't cover it. Then every sentence must carry a valid citation or it's removed.

Can the AI answer from its own knowledge?

No. The proof: turn grounding off and search returns nothing, so every question is refused. If it were using its own memory, it would still answer.

What if Gemini is down?

Answers switch to quoting the source passage word for word, clearly labelled. Date-Shift facts don't use AI at all, and generated briefings are cached, so the demo path is instant.

Why Gemini?

Fast, cheap, follows source-only instructions well, and has a free tier. The design isn't tied to it: the model sits behind one function, with backup models configured.

How did you pick the relevance threshold?

We measured it: 56 test questions (37 real, 19 trick, English and Hindi), and picked 0.30 as the score that best separates them. The results are in our calibration notes.

How do Hindi and Kannada work?

The languages come from the database (English, Hindi, and the city's own language). Search uses a multilingual model, so Hindi questions still match, and Sarvam AI translates the answers and the page.

About data and tech

Where does the data come from?

Only the PS-13 database the organisers provided: 60 cities, 900 places, 300 hotels, 300 events, 3,600 weather days, 400 advisories, and the city guides. We open it read-only.

What if the data is wrong?

Then you can see exactly which row it came from. Every chip opens the source row, so an error is easy to trace and fix at the source. That's the point of citations.

How do you add a new city?

Add its rows to the database and rebuild the search index. No code changes; nothing is hardcoded to a city.

Why this tech stack?

Python keeps the AI and API together, SQLite read-only protects the data, React makes the chips and drawers reusable, and it all runs free on one laptop.

How is it deployed?

The web app is on Vercel; the backend runs on our laptop and is exposed through a Cloudflare tunnel. For production we have a Docker setup for Render ready. It needs a 2 GB plan because the search model uses about 860 MB.

How would it scale to real users?

Move the backend to a cloud server, keep caching briefings per city, date and language, and add a managed database. Date-Shift is plain queries, so it's cheap to scale.

How do you test it?

68 automated tests (pytest) cover the Date-Shift facts, "What changed", citation lookups, refusals and the data rules, plus a 56-question evaluation set for the AI.

What happens with no internet?

The app saves every response in the browser. If the connection drops, an amber bar appears and saved briefings and places still show. New questions need a connection.

How fast is it?

Date-Shift facts are instant. A briefing is one AI call; once generated it's cached, so repeat views are instant. The chat usually answers in a few seconds.

About privacy, business and next steps

Why don't you collect user information?

We don't need it. Location is used only to pick the nearest city and isn't stored; there's no login. Nothing kept means nothing to leak, and no consent forms before a traveller gets help.

How would you make money?

License it to tourism boards, hotels and travel companies as a verified, cited city guide under their brand. Trust is the product.

What would you build next?

Multi-day trip plans that Date-Shift across the whole trip, live data sources behind the same citation system, and an installable phone app with a full offline mode.

What was the hardest part?

Making the AI refuse at the right time: not too often, not too rarely. Some trick questions look very similar to real ones, which is why we added rules before the similarity check and measured the threshold.

Honest limits

Say these confidently if asked. Knowing your limits is a strength.

- **Fixed data window:** the data covers 1 Sep to 30 Oct 2026 only; other dates move to the nearest one we have.
- **No live data by design:** no fares, traffic or live prices. We refuse rather than guess.
- **Hosting:** the backend runs locally. A Dockerfile and Render blueprint are included for a cloud setup, which needs about 2 GB of memory for the search model.
- **Travel times are estimates** (about 12 km/h), not live routing.
- **The map is a radar** drawn from coordinates, not street maps. Nothing is fetched from a map provider.
- **Translation needs Sarvam:** if it fails, English is shown.
- **The grounding switch is shared** by everyone using the same backend. It's a demo control.

If something breaks during a demo

Stay calm, say one line, switch to the fallback. Never debug in front of the panel.

WHAT HAPPENS	WHAT TO DO	WHAT TO SAY
Live site shows "offline"	Switch to the localhost:5173 tab	"Here it is running locally."
Location prompt denied	Tap the Bengaluru city card	"You can also just pick a city."
An answer is slow	Use the cached demo dates (25 Sep, 17 Oct, 24 Oct) and the FAQ answers	"Briefings are cached, so this is instant."

WHAT HAPPENS	WHAT TO DO	WHAT TO SAY
"AI model offline" banner	Keep going; answers are quoted from sources	"Even without the AI, we quote the source instead of guessing."
Wi-Fi drops	Keep going on saved data, or use the phone hotspot	"This is the offline cache working."
Everything fails	Play the backup recording	"Here's the recorded run of the same flow."

Jargon, explained

If a word comes up in a question, here's what it means in plain English.

Frontend

The part you see and tap: the web pages (our React app).

Backend

The program on a server that fetches data and talks to the AI (our FastAPI app).

API / endpoint

A web address the frontend calls to get data, like `/briefing`.

JSON

The simple text format the backend uses to send data to the app.

Database (SQLite)

Organised tables of data stored in one file. "Read-only" means we can read but never change it.

LLM (large language model)

An AI that writes text, like Gemini or ChatGPT.

Hallucination

When an AI states something false as if it were true.

Grounding

Forcing the AI to answer only from given sources, so every claim is backed.

RAG

Retrieval-Augmented Generation: find the relevant text first, then let the AI write from it. Like an open-book exam.

Embedding

Turning text into a list of numbers so that similar meanings get similar numbers.

Vector database (ChromaDB)

A database that finds text by meaning using embeddings, not by matching exact words.

Similarity score / threshold

How close a passage is to the question (0 to 1). Below our cut-off (0.30) we refuse.

Passage / chunk

A short piece of the city guide that can be searched and cited on its own.

Citation / source label

The tag after a sentence, like `weather_daily / wth_833a28a0`, naming the exact row or passage.

Prompt

The instructions and text we send to the AI.

Refusal

When GeoGuide politely says it can't answer from its sources, instead of guessing.

Deterministic

Always gives the same result for the same input. Our Date-Shift facts are deterministic; AI text is not.

Cache

A saved copy of a result so it doesn't need to be computed again. Faster and cheaper.

Offline cache (localStorage)

Saved copies kept inside your browser, used when there's no internet.

Geolocation

The browser feature that shares your coordinates (latitude and longitude) with permission.

Haversine distance

The formula for straight-line distance between two points on a globe. We use it to find the nearest city and places.

HTTPS

A secure web connection. Browsers only share location over HTTPS.

CORS

A browser safety rule; our backend lists which websites may call it (our Vercel site).

Deploy

Putting the app online so others can use it.

Vercel

The free hosting service running our web app.

Cloudflare tunnel

A secure public web address that forwards to a program on our laptop.

API key / environment variable

A secret password for a service (Gemini, Sarvam), kept in a private settings file, never in the code.

pytest / tests

Small programs that check our code still works correctly after every change.

Docker

A way to package the app with everything it needs so it runs the same on any server.